About this PDF

Built only from: **NOC:Natural Language Processing (NPTEL course book, weeks 5, 6)** — https://nptel.ac.in/courses/106105158

Licence: CC BY-SA 4.0. Accessed 2026-09-26.

Covers 5 of 11 syllabus sub-topics of this unit.

Partly covered in the source:

- Syntactic Parsing, Ambiguity

Not in the source (not covered in this PDF):

- Grammar rules for English, Treebanks
- DynamicProgramming parsing
- Shallow parsing
- Probabilistic CYK,Probabilistic Lexicalized CFGs
- Feature structures
- Unification of feature structures.

Context-Free Grammars

Definition

A context free grammar consists of a set of non-terminals, terminals, a start symbol and a set of production rules that rewrite a non-terminal as a sequence of terminals and non-terminals

Explanation

In a CFG each rule specifies how a constituent can be expanded. For a given non-terminal the sum of the probabilities of all its possible expansions must be 1 when probabilities are attached, forming a probabilistic CFG. This constraint ensures that the rule probabilities form a proper distribution over the alternatives for that non-terminal.

Example

Consider the simple grammar where the start symbol S expands to NP VP, and VP can expand either to V NP or to VP PP. The rule S → NP VP has probability 1, the two VP rules have probabilities 0.7 and 0.3 respectively, and the remaining rules each have probability 1. All probabilities for each non-terminal add up to 1, satisfying the required constraint.

Key points

- A CFG is defined by non-terminals, terminals, a start symbol and production rules
- When probabilities are added, the grammar becomes a probabilistic CFG
- For each non-terminal the probabilities of all its productions must sum to 1
- The probabilities allow computation of the likelihood of individual parse trees

Normal Forms for grammar

Definition

A grammar is in Chomsky normal form when every production rule has either exactly two non-terminals on the right-hand side or a single terminal. This restriction is used by the CKY algorithm to limit derivations to pairs of non-terminals or single terminals.

Explanation

Because each rule in CNF expands to at most two non-terminals, the CKY parser can fill a parsing table bottom-up by considering only binary splits of a span. For a word span x i j the parser looks for a rule whose right-hand side matches two non-terminals that derive the two sub-spans, or a rule that directly derives the single terminal in the diagonal cells. This property allows systematic construction of all possible non-terminals that can generate any substring of the sentence.

Example

Consider the sentence “a pilot likes flying planes”. When trying to derive the span x0 3 (“a pilot likes”), the parser checks the two possible binary splits: x0 1 with x1 3, and x0 2 with x2 3. It finds that no rule in the CNF grammar produces the required pair of non-terminals for either split, so the cell remains empty. Similar checks are done for other spans such as x1 4 (“pilot likes flying”).

Algorithm / Steps

1. List the words w1 … wn of the input sentence and imagine lines 0 … n separating them.
2. Create a triangular table where each cell x i j will hold all non-terminals that can derive the words between line i and line j.
3. Fill the diagonal cells x i i+1 with non-terminals that have a rule producing the single terminal wi.
4. For each span length greater than 1, compute x i j by checking all possible split points k ($i < k < j$). If there is a rule $A \rightarrow B\ C$ with B in x i k and C in x k j, add A to x i j.
5. Continue until the cell x0 n is filled; if the start symbol appears there, the sentence is generated by the grammar.

Key points

- CNF restricts productions to two non-terminals or one terminal, enabling binary parsing.
- CKY builds a table where each cell records non-terminals that can generate the corresponding word span.
- Diagonal cells are filled using terminal rules; other cells are filled by combining results from smaller spans.
- The algorithm proceeds bottom-up, considering all possible binary splits for each span.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Dependency Grammar

Definition

Dependency grammar represents a sentence as a set of binary relations between words, where each relation links a **head** (or governor) to a **dependent** (or modifier). The structure is formalized as a directed graph whose nodes are the word forms and whose labeled arcs denote the grammatical relation (e.g., subject, object) between a head and its dependent.

Explanation

In a dependency structure the head determines the syntactic category of the whole construction, and the dependent contributes a particular grammatical function such as subject, object, or modifier. The graph must satisfy several linguistic constraints:

- **Single-head constraint** – each word has at most one head;
- **Acyclicity** – the graph is hierarchical with no cycles;
- **Connectedness** – no node is isolated; every word is reachable via some arc;
- **Projectivity** – arcs must not cross; this constraint is language-dependent.

Example

Consider the simple sentence “Ram eats apple.” The verb *eats* is the main verb (head). *Ram* is linked to *eats* with a **subject** relation, and *apple* is linked to *eats* with an **object** relation.

Key points

- Dependency parsing directly captures binary head-dependent relations between words, unlike constituency parsing which groups words into phrases.
- Arrows in the diagram point from head to dependent and are labeled with grammatical functions such as subject or object.
- The dependency graph is a directed, labeled graph that must be connected, acyclic, single-headed, and (often) projective.

Syntactic Parsing, Ambiguity

Partly covered in the source.

Definition

Syntactic parsing aims to produce a dependency graph for a sentence, where each word is linked to a single syntactic head and the graph satisfies constraints such as connectedness, acyclicity, single-head, and projectivity.

Example

Consider a sentence where a dependency link runs from word a to word b. A crossing link from b to c (with c on the opposite side of a) creates a non-projective configuration because the edges intersect. In a projective configuration, such crossing would not occur, and all links would respect the side-consistency rule.

Key points

- Every word must have exactly one syntactic head (single-head constraint).

- Dependency graphs must be connected and acyclic, forming a hierarchical structure.
- Projectivity prohibits crossing edges; violations are called non-projective dependencies.
- Some languages (e.g., English) typically obey projectivity, while others may not, influencing the choice of parsing algorithm.

Probabilistic CFG

Definition

Probabilistic context free grammars (PCFG) are a simple extension of context free grammars in which each rule is assigned a probability. The grammar consists of the same components as a CFG (terminals T, non-terminals N, start symbol S, and rules R) together with a probability function P that gives a probability distribution over the rules. For every non-terminal X, the probabilities of all rules with left-hand side X must sum to 1.

Explanation

A PCFG allows us to compare different parses of the same sentence by computing the probability of each parse tree. By multiplying the probabilities of the rules used in a tree, we obtain the tree's probability, which provides a measure of plausibility for that parse. This enables us to identify the more likely parse among many possible trees generated by a CFG. The probability of a whole sentence can be obtained by summing the probabilities of all its parse trees, although this does not capture lexical co-occurrence and is generally less suitable than a language model for sentence probability estimation.

Example

Consider the sentence "book the dinner flight". Using a PCFG we can compute probabilities for two possible parse trees: the first tree has probability 1.62 into 10 to the power minus 6, and the second tree has probability 2.28 into 10 to the power minus 7. Since the first probability is larger, the first parse is the more likely interpretation.

Key points

- Each rule in a PCFG carries a probability, and the probabilities for all rules expanding the same non-terminal sum to 1.
- The probability of a parse tree is the product of the probabilities of the rules used in that tree.
- Comparing parse tree probabilities lets us choose the most plausible parse for a given sentence.
- Sentence probability can be obtained by summing the probabilities of all its parse trees, but this measure ignores lexical co-occurrence and is less effective than a language model for evaluating sentence plausibility.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Source and licence

This PDF is adapted from **NOC:Natural Language Processing (NPTEL course book, weeks 5, 6)** (https://nptel.ac.in/courses/106105158), accessed 2026-09-26.

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).